Bitácora S09

Mensajería con RabbitMQ: que el aviso no dependa de que notificación esté despierta

Sistemas Distribuidos 2027-1 · Grupo 2 ·

Dato	Detalle
Alumna	Alejandra Melissa Saucedo Serralde
Práctica	Lab S09: la saga de transferencia publica un evento en RabbitMQ en lugar de llamar a `notificacion` por HTTP. Incluye la Parte 0, el publicador con confirmación, el consumidor con ack manual, la idempotencia por `eventoId` y la DLQ.
Fechas	Parte 0: 31 de octubre de 2026. El resto (publicar, consumir, duplicado y DLQ): 6 y 7 de noviembre de 2026.
Horas	Los logs de Docker y las fechas guardadas en la base están en UTC; mi hora local es UTC−6 (01:35 UTC = 19:35 local).
Reto opcional	No hice el Outbox (sección 7 del lab). Explico en la sección 6 qué pasa por no tenerlo.

Resultados de un vistazo

Qué probé	Resultado
Parte 0: `notificacion` apagada	La transferencia responde `500`, los saldos sí se mueven (950 y 550) y la fila queda a medias (no llega a `COMPLETADA`).
Publicar con `notificacion` apagada	`201` y estado `COMPLETADA` con su `eventoId`. Sin cola enlazada, el evento vuelve con `NO_ROUTE` y queda en el log.
Desacople en tiempo	Tres transferencias con `notificacion` apagada: tres `201` y la cola con 3 `Ready`. Al volver, la cola baja a 0 y llegan los avisos.
Transferencia revertida	`502` con `COMPENSADA`, saldo de Ana igual que antes y un aviso nuevo `DEVOLUCION_TRANSFERENCIA por 150`.
Duplicado	Antes: dos avisos `por 20`. Después: un solo aviso y, en el log, `ya procesado: se ignora` y `0 aviso(s) nuevo(s)`.
DLQ	El mensaje `{"hola":` termina en `notificacion.transferencias.dlq` con `x-death` (`reason: rejected`). Las transferencias normales siguen generando avisos.
Pruebas unitarias	`transferencia`: 18 pruebas (10 de `TransferenciaServiceTest`), 0 fallas. `notificacion`: 3 pruebas, 0 fallas.

1. El problema

Antes de tocar código reproduje la Parte 0. Dejé `notificacion` apagada con `docker compose stop notificacion` y mandé una transferencia de 50 de Ana (`...0001`) a Beto (`...0002`). Repetí la transferencia una segunda vez para tener la evidencia en una sola captura.

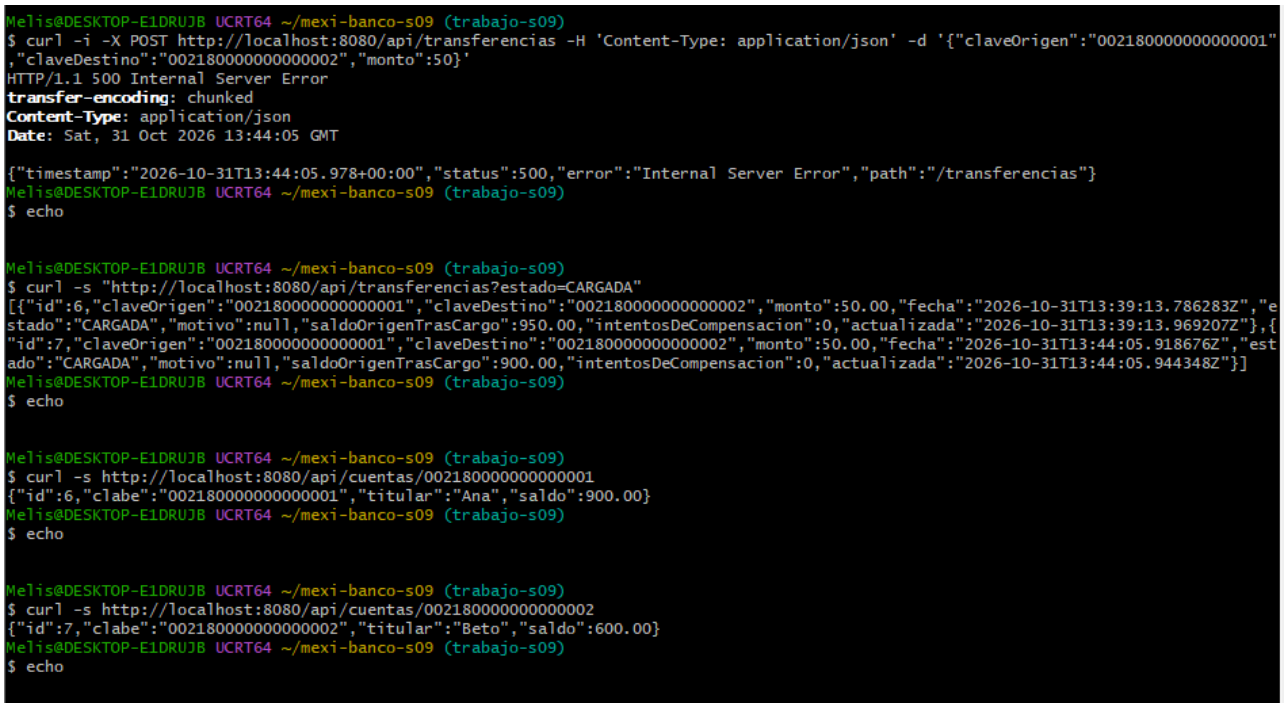


Figura 1. *Parte 0: la transferencia responde `500 Internal Server Error`; la consulta por estado muestra las filas 6 y 7 sin terminar (`CARGADA`); y los saldos sí cambiaron: Ana 900 y Beto 600 (dos transferencias de 50).*

Lo que se ve: el cliente recibe un error, pero el dinero **sí se movió** (Ana bajó de 1000 a 900 y Beto subió de 500 a 600). La fila de cada transferencia se quedó a medias y nadie la termina, ni siquiera cuando vuelve a encenderse `notificacion`.

> **Por qué pasó, en una frase:** el cargo, el abono y los asientos ya estaban hechos y guardados, pero la saga todavía tenía que llamar a `notificacion` por HTTP y **esperar su respuesta** antes de marcar la transferencia como `COMPLETADA`; con `notificacion` apagada esa espera falló, el error subió hasta el cliente y la transferencia se quedó a medias.

Una observación sobre esta captura

El lab dice que esta fila debería quedar en `ABONADA` y que el error sería un `503` (Connect timed out) o un `500` (Service Instance cannot be null) por la llamada a `notificacion`. En mi corrida la fila quedó en `CARGADA`. Más adelante descubrí por qué: la base de `transferencia` ya existía de la S08 y tenía un constraint `transferencia_estado_check` que no incluía el estado `ABONADA`; al intentar guardarlo, Postgres lo rechazó y la saga se detuvo **antes** de llegar a la llamada a `notificacion`. El detalle de ese error y su arreglo está en el Anexo A. Lo que muestra la captura (error al cliente, dinero movido, fila sin terminar) es el mismo síntoma, y la causa que explica el lab (esperar a `notificacion` de forma síncrona dentro de la saga) la comprobé leyendo el código de `transferir()`: las llamadas `notificaciones.notificar(...)` están antes de `marcarCompletada()`.

2. Publicar el evento

En lugar de llamar a `notificacion`, `transferencia` ahora **deja un recado** en el exchange `transferencias` de RabbitMQ y sigue con su trabajo. No sabe quién lo va a leer ni si está despierto. El cambio tiene seis piezas.

2.1 Cada transferencia nace con su eventoId

El `eventoId` se genera **cuando nace la fila** y se guarda con ella, no cuando se publica. Así, si el mismo evento se publica dos veces, lleva el mismo id, y quien lo recibe puede reconocerlo como repetido. Si lo generara al publicar, cada reintento parecería un evento nuevo.

```
@Column(unique = true)
private UUID eventoId;

// en el constructor:
this.eventoId = UUID.randomUUID();

// las filas de la S08 no traen eventoId: se les pone uno al leerlas
@PostLoad
void asegurarEventoId() {
        if (eventoId == null) { eventoId = UUID.randomUUID(); }
}
```

2.2 El evento es un contrato propio

El evento es un `record` aparte (`EventoTransferencia`) y no la entidad `Transferencia`: es el **contrato** con quien escuche, así que cambiar una columna de mi tabla no debe romper a nadie afuera. Es un hecho en pasado ("ya se hizo"), no una orden. Tiene dos tipos, `TransferenciaRealizada` y `TransferenciaRevertida`, con su routing key (`transferencia.realizada` y `transferencia.revertida`).

2.3 El exchange y la confirmación del broker

`transferencia` solo declara el **exchange** (`topic` y durable) y publica ahí. Las **colas** no son suyas: son de quien consume. En el `application.yml` activé tres cosas del publicador:

```
spring:
  rabbitmq:
    connection-timeout: 2s              # si el broker no contesta, rendirse a los 2 s
    publisher-confirm-type: correlated  # el broker contesta ack o nack por cada mensaje
    publisher-returns: true             # y regresa el que ninguna cola recibio...
    template:
      mandatory: true                   # ...en lugar de tirarlo en silencio
```

2.4 El publicador

`PublicadorDeEventos.publicar()` manda el evento con entrega **persistente** (se escribe a disco), lo ata a una `CorrelationData` con su `eventoId` y **espera la confirmación** del broker. Si el broker no confirma, lo rechaza (`nack`) o ninguna cola recibió el mensaje (`mandatory`), lanza `EventoNoPublicadoException`.

```
rabbit.convertAndSend(EXCHANGE, evento.routingKey(), evento, mensaje -> {
    mensaje.getMessageProperties().setDeliveryMode(MessageDeliveryMode.PERSISTENT);
    mensaje.getMessageProperties().setMessageId(id);
    return mensaje;
}, correlacion);

var confirmacion = correlacion.getFuture().get(esperaConfirmacionMs, TimeUnit.MILLISECONDS);
if (!confirmacion.isAck()) { throw new EventoNoPublicadoException("...nack..."); }
if (correlacion.getReturned() != null) { throw new EventoNoPublicadoException("Ninguna cola
recibio..."); }
```

2.5 La saga publica después de guardar

Quité las dos llamadas `notificaciones.notificar(...)` de `transferir()`. Ahora la saga marca `COMPLETADA`, la **guarda** y solo entonces publica:

```
t.marcarCompletada();
t = guardar(t);                          // primero la base dice COMPLETADA
avisar(EventoTransferencia.realizada(t.getEventoId(), t.getId(), claveOrigen, claveDestino,
monto,
        t.getSaldoOrigenTrasCargo(), destino.saldo(), t.getActualizada()));   // luego el
recado
return t;
```

El orden importa: al revés, `notificacion` podría avisar de una transferencia que la base todavía no dice que pasó. Y en `compensar()`, cuando la transferencia queda `COMPENSADA`, se publica `TransferenciaRevertida` (también cuando la termina el reintento programado de la S08, porque pasa por el mismo método).

Si `publicar` falla, `avisar()` **no revierte** la transferencia: el dinero ya se movió bien y el cliente recibe su `201`. Solo se pierde el aviso y queda registrado en el log con un `ERROR`.

```
private void avisar(EventoTransferencia evento) {
        try {
                eventos.publicar(evento);
                log.info("Transferencia {}: evento {} publicado ({})", ...);
        } catch (RuntimeException sinPublicar) {
                log.error("Transferencia {}: quedo terminada pero el evento {} {} NO se
publico: {}", ...);
        }
}
```

2.6 Prueba con notificacion apagada

Con `notificacion` apagada (y todavía sin que nadie hubiera declarado una cola) hice una transferencia de 50. Responde `201` y queda `COMPLETADA`, con su `eventoId`, aunque `notificacion` no esté:

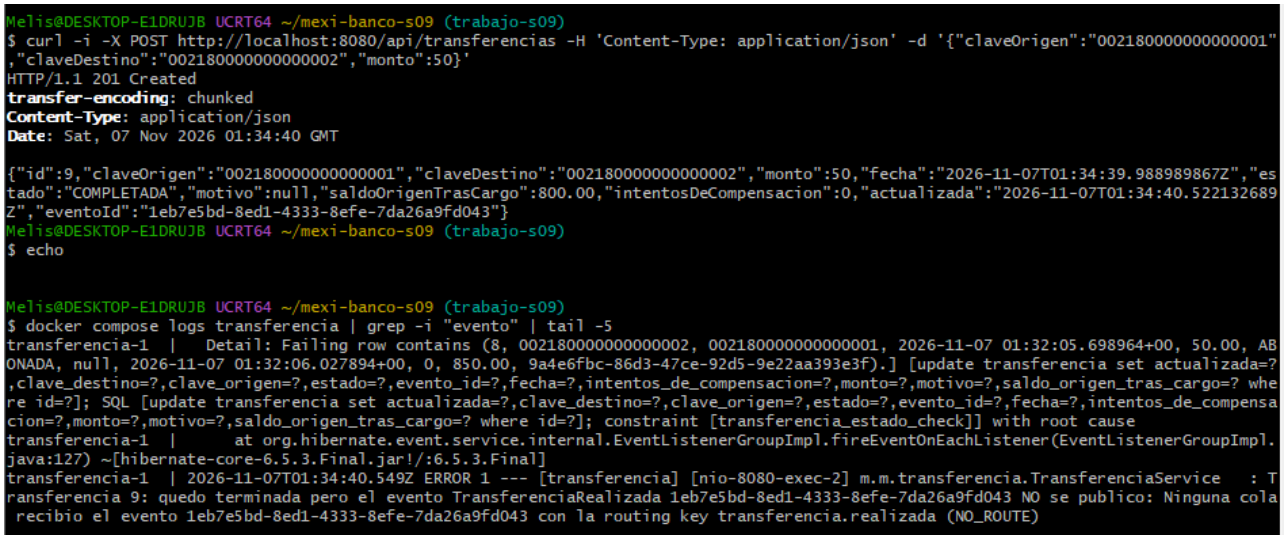


Figura 2. *`201 Created` con `estado: COMPLETADA` y `eventoId` (transferencia 9), con `notificacion` apagada.*

En el log de `transferencia` el evento aparece como **no publicado**, porque ninguna cola lo recibió (`NO_ROUTE`): lo regresó el broker gracias a `mandatory`.

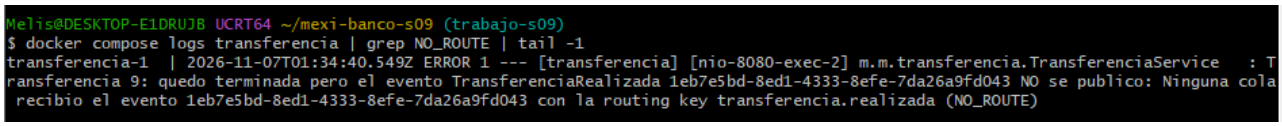


Figura 3. *Log de `transferencia`: el evento `TransferenciaRealizada` no se publicó porque ninguna cola lo recibió (`NO_ROUTE`).*

En el panel, el exchange `transferencias` existe como `topic` y durable, pero la sección **Bindings** está vacía: nadie había declarado una cola enlazada. Un exchange no guarda mensajes, así que ese aviso se perdió.

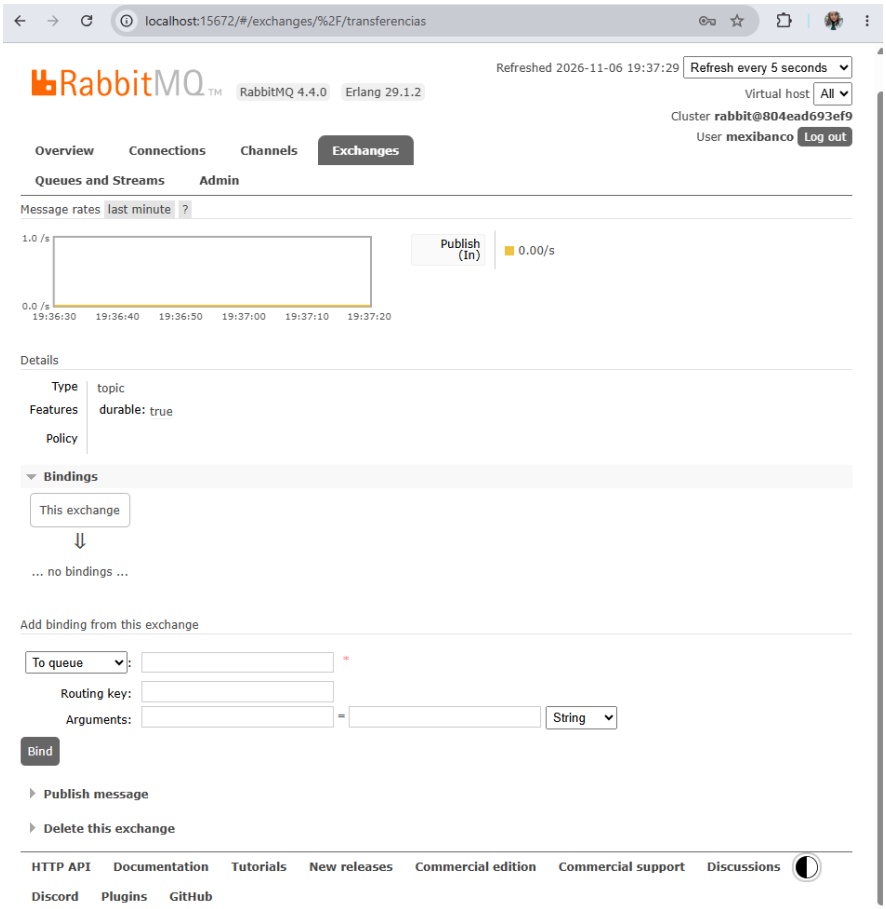


Figura 4. *Exchange* `transferencias`*: tipo* `topic`*,* `durable: true` *y* `no bindings`*.*

Qué garantiza el publisher confirm (y qué no)

> **Lo que me garantiza:** que el broker recibió el mensaje y lo aceptó (`ack`), y con `mandatory`, además, que se enrutó a alguna cola. Si algo de eso falla, me entero y lo dejo en el log. **Lo que no me garantiza:** que alguien lo haya leído ni procesado. Si `notificacion` está apagada, el mensaje queda en la cola esperando, sin perderse, **siempre que la cola exista**. Y tampoco cubre el hueco entre `guardar(t)` y `avisar(...)`: si `transferencia` muere justo ahí, la base ya dice `COMPLETADA` pero el evento nunca se publicó, y no queda nada que lo reintente.

3. Desacople en tiempo

Ahora el otro lado del buzón. `notificacion` necesita tres cosas: su **cola** enlazada al exchange, un **consumidor** que lea cada mensaje y guarde los avisos, y una regla: confirmar el mensaje **solo después de guardar**. Antes de empezar, el panel estaba limpio: solo los exchanges que trae RabbitMQ y ninguna cola.

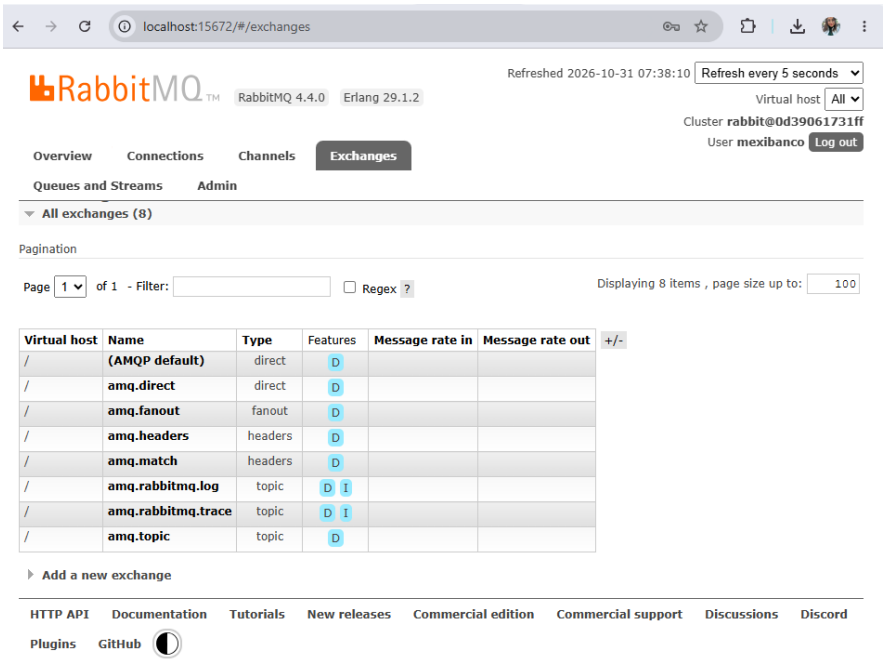
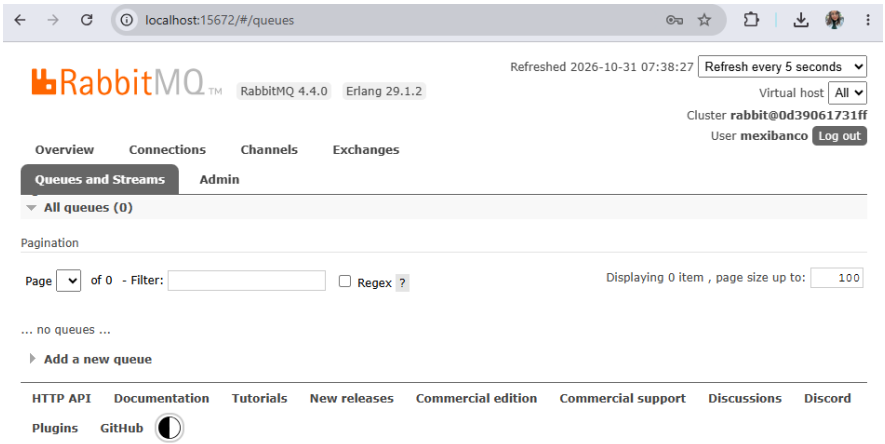


Figura 5. *Panel antes de empezar, pestaña Exchanges: solo los 8 exchanges que trae RabbitMQ.*



Zoom

Figura 6. *Panel antes de empezar, pestaña Queues and Streams:* `no queues`.

3.1 La cola es de notificacion

`notificacion` declara su propia cola durable, `notificacion.transferencias`, enlazada al exchange con el patrón `transferencia.*`. Se declara al arrancar el servicio: sin ese arranque la cola no existe y los mensajes se pierden como vimos en la sección 2.

```
public static final String EXCHANGE = "transferencias";
public static final String COLA = "notificacion.transferencias";
public static final String PATRON = "transferencia.*";

Queue cola = QueueBuilder.durable(COLA).build();   // durable: sobrevive a un reinicio del
broker
return new Declarables(transferencias, cola,
        BindingBuilder.bind(cola).to(transferencias).with(PATRON));
```

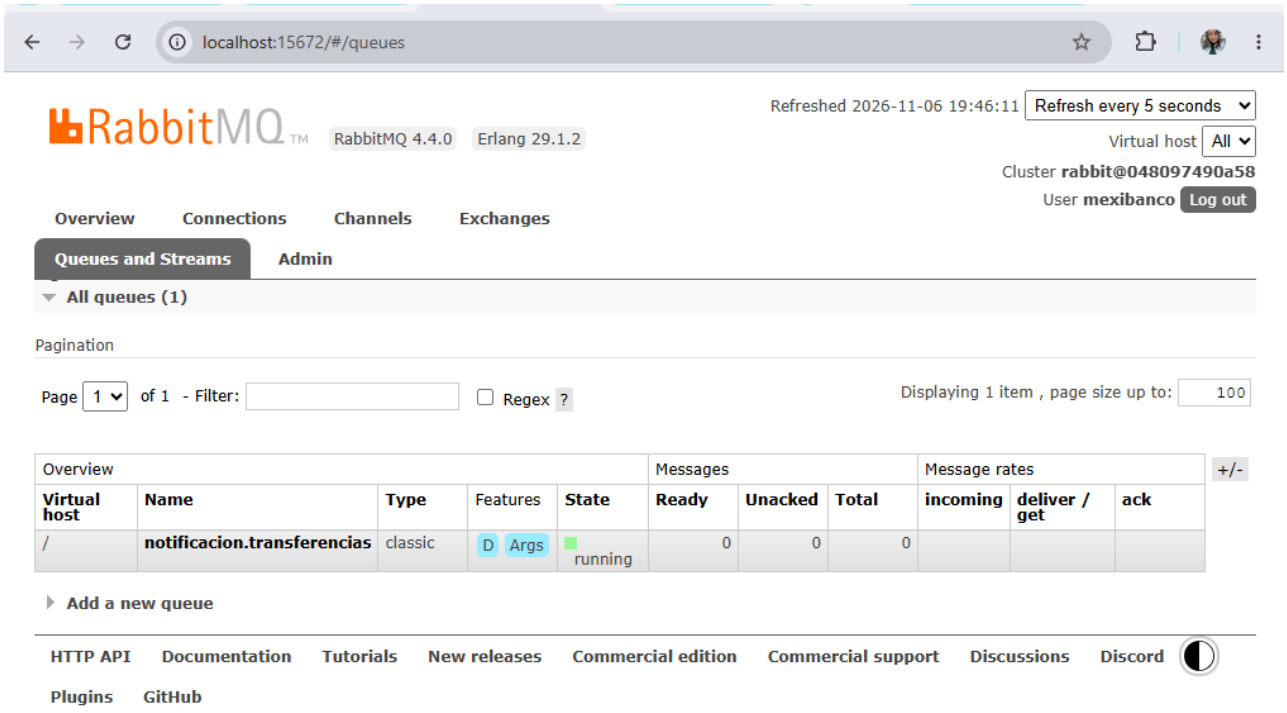


Figura 7. *Al arrancar `notificacion` aparece la cola `notificacion.transferencias` (durable, `running`, sin mensajes).*

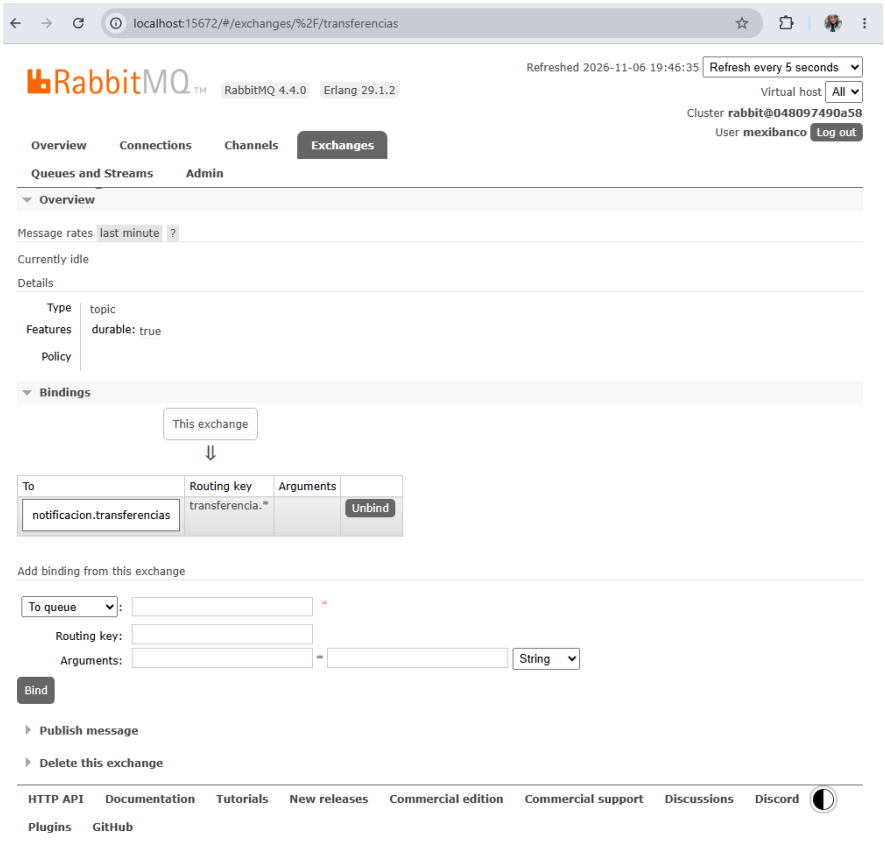


Figura 8. *Exchange `transferencias`, sección Bindings: la cola `notificacion.transferencias` enlazada con la routing key `transferencia.*`.*

3.2 Guardar los avisos y confirmar después

Cada `TransferenciaRealizada` genera **dos** avisos (a quien envió y a quien recibió) y cada `TransferenciaRevertida`, **uno** (a origen, porque su dinero regresó). `NotificacionService.avisarDeTransferencia()` los guarda en una sola transacción: se guardan los dos o ninguno. El consumidor usa **ack manual** y confirma **después** de guardar:

```
# application.yml de notificacion
spring:
  rabbitmq:
    listener:
      simple:
        acknowledge-mode: manual   # el consumidor dice cuando (basicAck)
        prefetch: 5                # maximo 5 sin ack a la vez

List<Notificacion> avisos = notificaciones.avisarDeTransferencia(evento);   // aqui hace
commit
log.info("[consumidor] {} de la transferencia {} (evento {}, redelivered={}): {} aviso(s)
nuevo(s)", ...);
canal.basicAck(etiqueta, false);   // DESPUES de guardar: "ya lo tengo, borralo"
```

3.3 El experimento: tres transferencias con notificacion apagada

Apagué `notificacion` y mandé tres transferencias de 11, 12 y 13. Las tres respondieron `201`, y la cola `notificacion.transferencias` quedó con **3 mensajes Ready, 0 sin confirmar**.

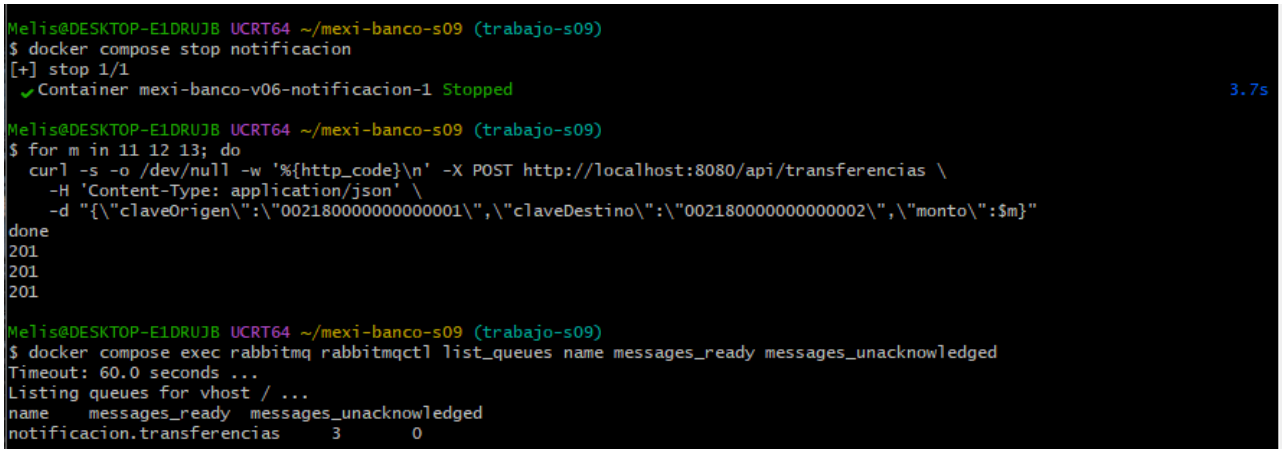


Figura 9. *`notificacion` detenida, tres `201` y `rabbitmqctl list_queues: notificacion.transferencias` con 3 `Ready` y 0 `Unacked`.*

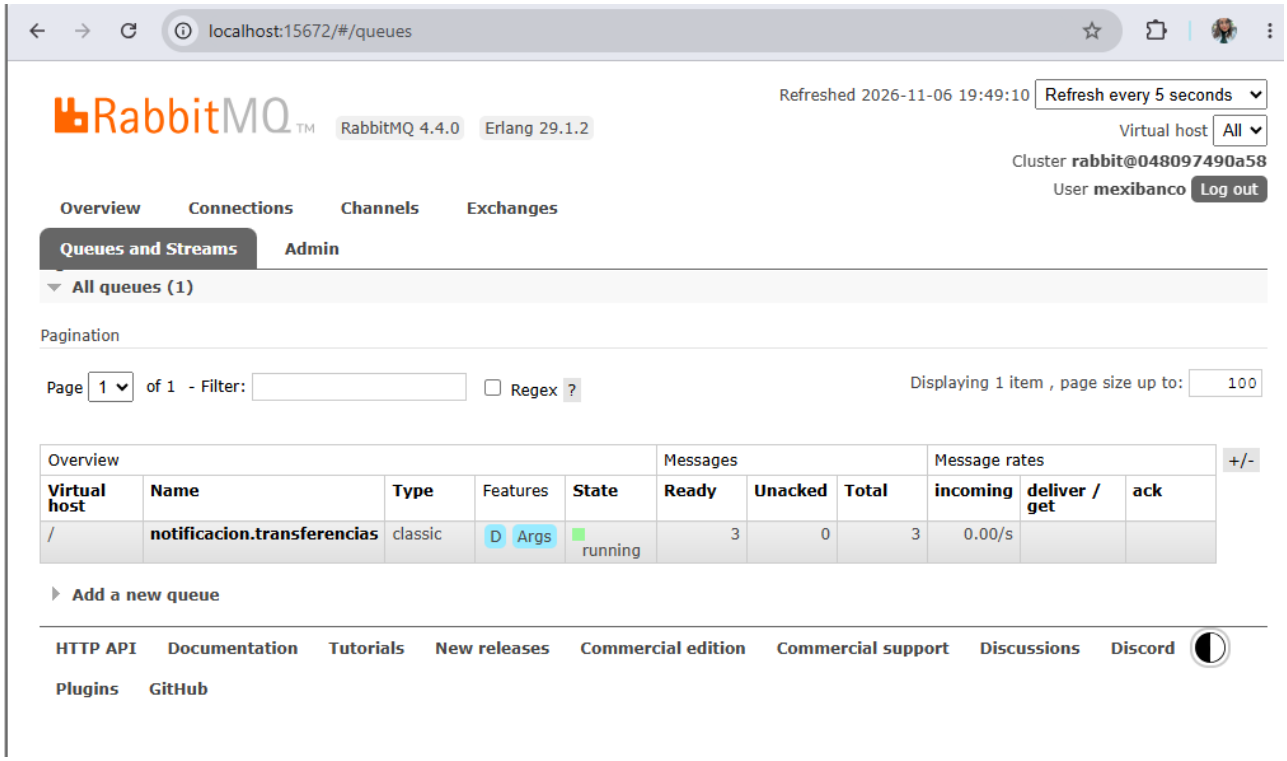


Figura 10. *El mismo estado en el panel:* `Ready = 3`, `Unacked = 0`, `Total = 3`, *sin consumidores.*

Al encender `notificacion`, la cola bajó a **0**, el log mostró un `[consumidor]` por evento (`redelivered=false`, 2 avisos nuevos cada uno) y la bandeja de Beto recibió los tres `TRANSFERENCIA_RECIBIDA`. Esta es la salida de la terminal:

```
$ docker compose exec rabbitmq rabbitmqctl list_queues name messages_ready
messages_unacknowledged
notificacion.transferencias     0       0

$ curl -s 'http://localhost:8080/api/notificaciones?clabe=002180000000000002'
[{"id":13,...,"mensaje":"TRANSFERENCIA_RECIBIDA por 13, saldo resultante 736.00"},
 {"id":11,...,"mensaje":"TRANSFERENCIA_RECIBIDA por 12, saldo resultante 723.00"},
 {"id":9, ...,"mensaje":"TRANSFERENCIA_RECIBIDA por 11, saldo resultante 711.00"}, ...]

[consumidor] TransferenciaRealizada de la transferencia 10 (evento 1e3cdc39-...,
redelivered=false): 2 aviso(s) nuevo(s)
[consumidor] TransferenciaRealizada de la transferencia 11 (evento 4098b676-...,
redelivered=false): 2 aviso(s) nuevo(s)
[consumidor] TransferenciaRealizada de la transferencia 12 (evento 81629f23-...,
redelivered=false): 2 aviso(s) nuevo(s)
```

Eso es el **desacople en tiempo**: el productor y el consumidor ya no tienen que estar vivos al mismo tiempo. Las transferencias se completaron mientras `notificacion` estaba caída y los avisos llegaron cuando volvió, sin perder ninguno.

3.4 La transferencia revertida

Probé también la revertida con la falla provocada de la S08: `FALLAR_ABONO_A_CLABE=002180000000000002`, que hace fallar a propósito el abono hacia Beto, y una transferencia de 150 de Ana a Beto. La saga devuelve el cargo y publica `TransferenciaRevertida`.

Mi primer intento no salió como el lab. La respuesta fue `502`, pero con `estado: COMPENSACION_PENDIENTE` (transferencia 13), no `COMPENSADA`. El saldo de Ana ya estaba en 764 (la devolución sí llegó a `cuenta`), pero fallaba otro paso de la compensación: `movimiento` respondía `500` al registrar el asiento de la devolución. El log de `movimiento` dio la causa: un constraint viejo, `movimiento_tipo_check`, que no aceptaba el tipo `DEVOLUCION_TRANSFERENCIA` (Anexo A). Mientras tanto, el reintento programado de la S08 volvía a intentarlo cada 10 segundos y fallaba igual. Arreglé el constraint y, en la siguiente vuelta, **la propia saga terminó la compensación sola**: la fila pasó a `COMPENSADA` y se publicó el evento.

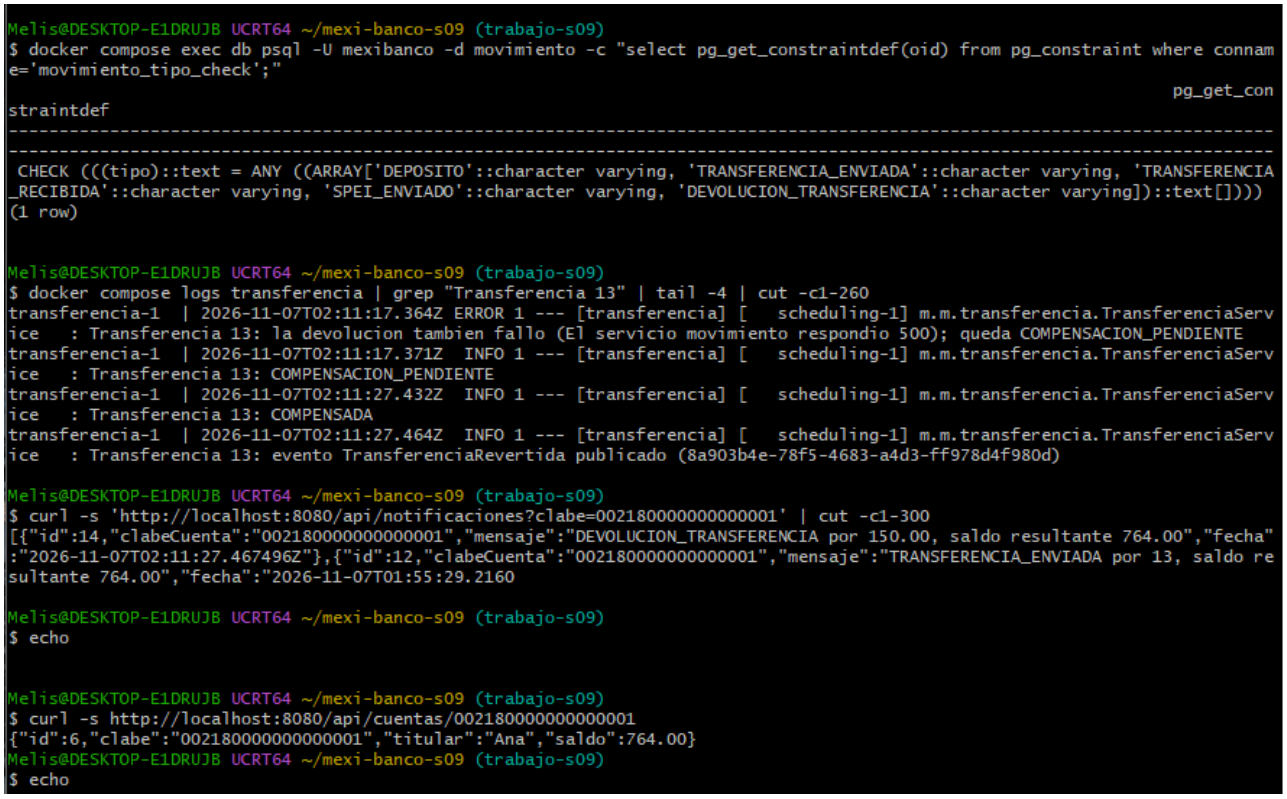


Figura 11. *Tras corregir el constraint: el log de `transferencia` muestra `COMPENSACION_PENDIENTE`, después `COMPENSADA` y `evento TransferenciaRevertida publicado`; en la bandeja de Ana llega `DEVOLUCION_TRANSFERENCIA por 150.00` y su saldo sigue en 764.00 (no se devolvió dos veces).*

Con el constraint ya arreglado repetí la prueba y salió lo que dice el lab: `502` con `estado: COMPENSADA` en un solo intento, el saldo de Ana igual que antes (764) y **un** aviso nuevo en su bandeja, `DEVOLUCION_TRANSFERENCIA por 150`. Hasta la S08 nadie le avisaba al cliente que su transferencia se había revertido; ahora sí.

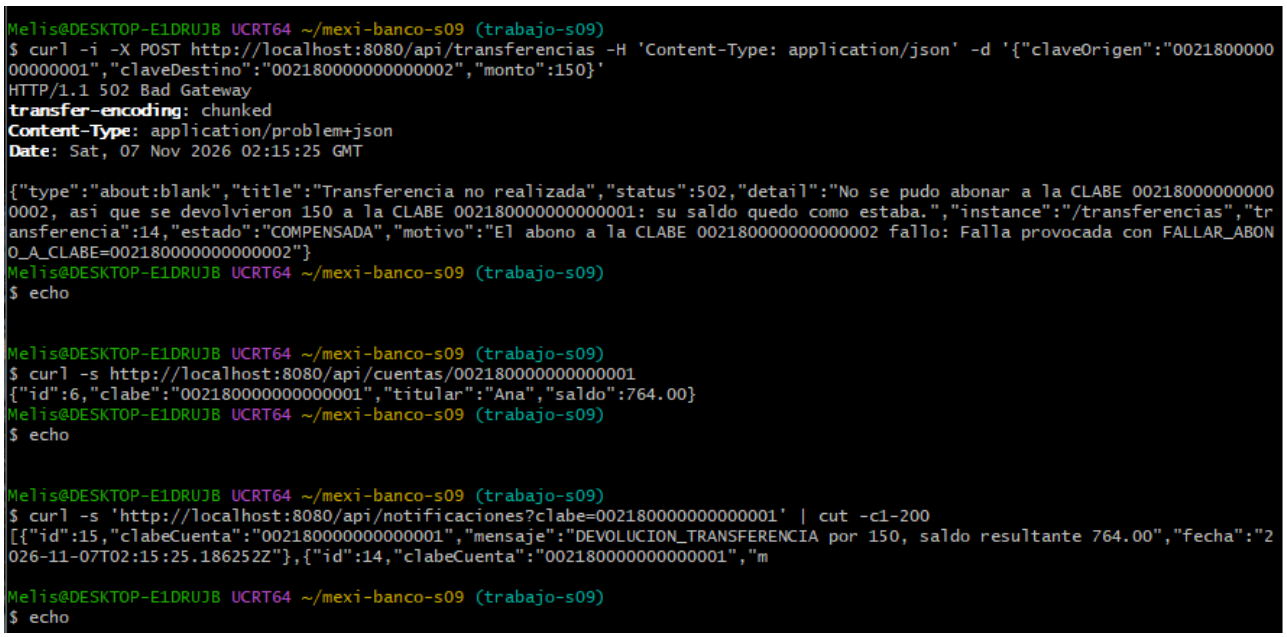


Figura 12. *502 con `estado: COMPENSADA` (transferencia 14), saldo de Ana en 764.00 y arriba de su bandeja `DEVOLUCION_TRANSFERENCIA por 150`.*

Esto se ve en la bitácora como una consecuencia de que la compensación de la S08 es **idempotente y reintentable**: aunque `movimiento` falló varias veces, la devolución del dinero se aplicó una sola vez (la clave de idempotencia `transferencia-13-devolucion` evitó el doble depósito) y el aviso salió una sola vez.

4. El duplicado

La mensajería entrega **al menos una vez**: un mismo mensaje puede llegar dos veces. Mi consumidor, tal como lo dejé en la sección 3, guarda un juego nuevo de avisos cada vez que recibe un evento, aunque sea el mismo. Primero lo provoqué y después lo arreglé.

4.1 Cómo lo provoqué (desde el panel)

- Apagué `notificacion` y mandé una transferencia de 20: la cola quedó con **1** mensaje `Ready`.
- En el panel, `notificacion.transferencias` → **Get messages** con **Nack message requeue true**: vi el mensaje sin consumirlo y copié su Payload (con su `eventoId`). El panel lo regresa a la cola, por eso el mensaje original sale con `redelivered=true`.
- En el exchange `transferencias` → **Publish message**, con la routing key `transferencia.realizada`, publiqué el **mismo cuerpo**, con el mismo `eventoId`. La cola quedó con **2** `Ready`.



Figura 13. *Mensaje original visto con* ***Get messages****: transferencia 15,* `eventoId 765a4aa3-...`*,* `Redelivered` *falso en la primera lectura.*

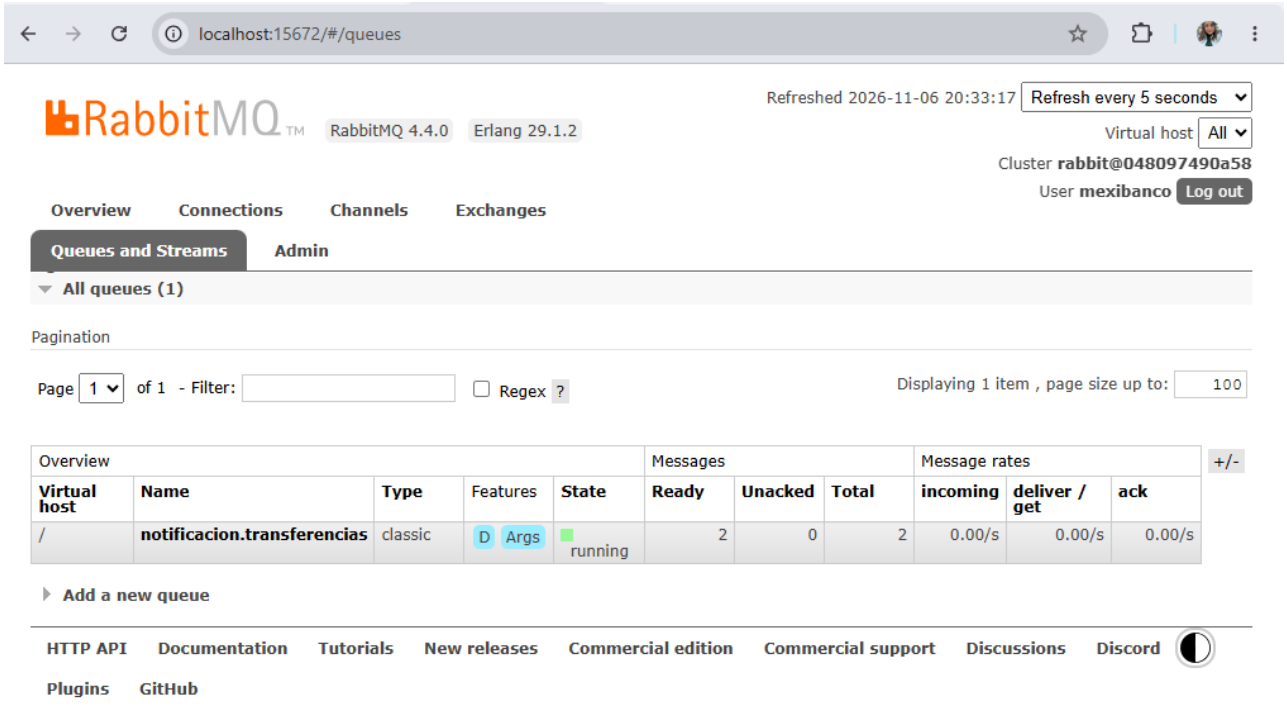


Figura 14. *Después de publicar la copia:* `notificacion.transferencias` *con 2* `Ready`*.*

4.2 El "antes": avisos repetidos

Encendí `notificacion`. El log muestra la **misma transferencia 15** (mismo `eventoId`) procesada **dos veces**, cada una con `2 aviso(s) nuevo(s)`, y la bandeja de Beto tiene **dos** `TRANSFERENCIA_RECIBIDA por 20` (ids 17 y 19) cuando debería haber uno.

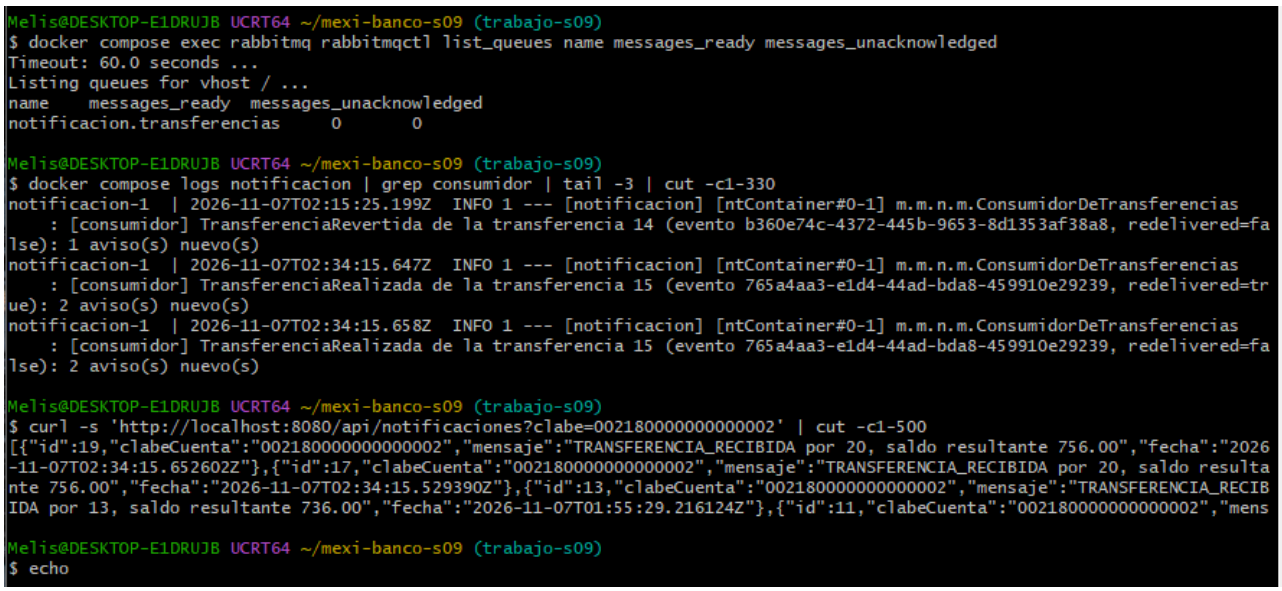


Figura 15. *Antes del arreglo: dos líneas `[consumidor]` de la transferencia 15 con 2 avisos nuevos cada una, y dos avisos `TRANSFERENCIA_RECIBIDA por 20` en la bandeja de Beto.*

4.3 El arreglo: idempotencia por eventoId

Guardé cada `eventoId` en una tabla (`evento_procesado`) con el `eventoId` como **llave primaria**, y lo hago **en la misma transacción** que los avisos. La marca es **una sola sentencia** de Postgres: `INSERT ... ON CONFLICT (evento_id) DO NOTHING`, que regresa cuántas filas insertó (1 si el evento era nuevo, 0 si ya estaba). Al principio de `avisarDeTransferencia`:

```
@Modifying
@Query(value = "insert into evento_procesado (evento_id, tipo, transferencia_id, fecha) "
        + "values (:id, :tipo, :transferencia, now()) on conflict (evento_id) do nothing",
nativeQuery = true)
int marcarSiEsNuevo(UUID id, String tipo, Long transferencia);

// dentro del mismo @Transactional que saveAll(avisos)
if (procesados.marcarSiEsNuevo(evento.eventoId(), evento.tipo(), evento.transferenciaId())
== 0) {
        log.info("[notificacion] evento {} ya procesado: se ignora", evento.eventoId());
        return List.of();   // 0 avisos nuevos, y el consumidor hace ack igual
}
```

Por qué así:

- **No** hice “consultar si existe y luego insertar”: entre las dos llamadas, otra entrega del mismo mensaje se puede colar. La sentencia única lo evita.
- **No** atrapo la violación de llave: en Postgres eso dejaría la transacción abortada. Con `ON CONFLICT DO NOTHING` no hay error.
- Va en **la misma transacción** que los avisos: si algo falla después de la marca, el rollback se lleva marca y avisos juntos, y la siguiente entrega lo intenta desde cero.

4.4 El "después"

Repetí el experimento con otra transferencia (de 30, transferencia 16): dejé 2 mensajes con el mismo `eventoId` en la cola y encendí `notificacion`.

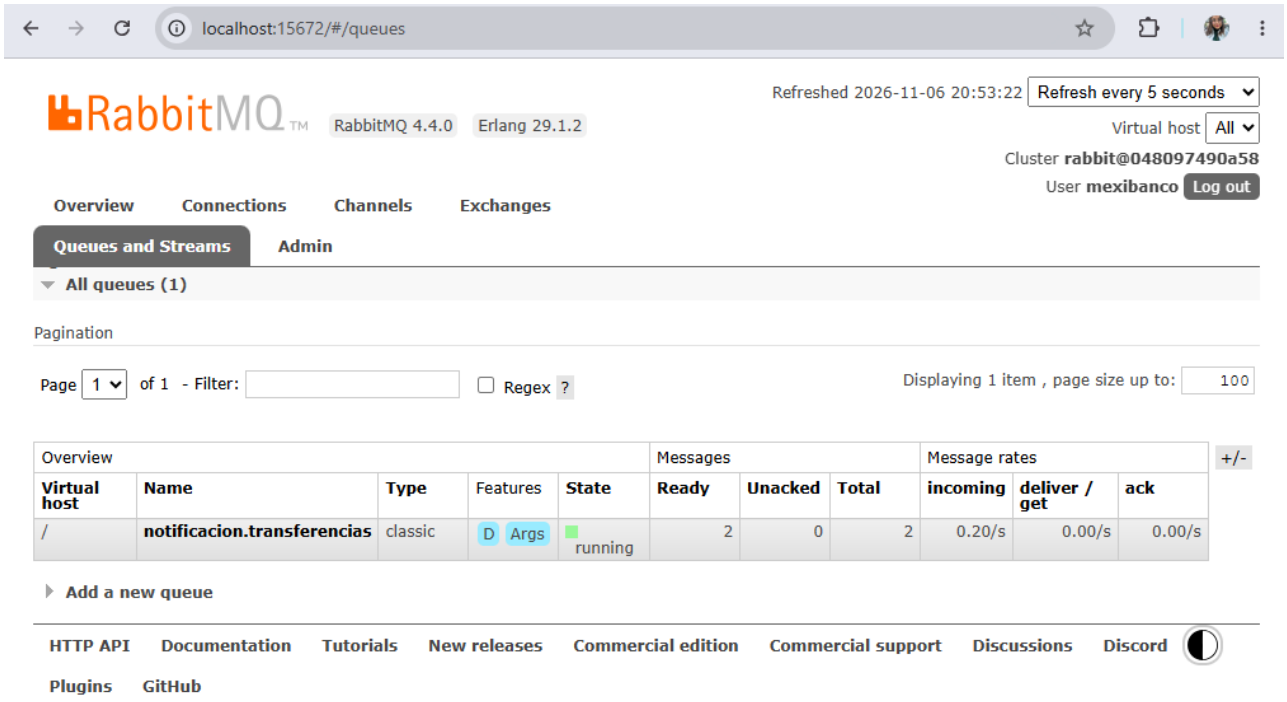


Figura 16. *El experimento repetido: la cola con 2* `Ready` *(original y copia con el mismo* `eventoId`*).*

En el log, la primera entrega guarda `2 aviso(s) nuevo(s)`, la segunda es reconocida (`ya procesado: se ignora`) y termina con `0 aviso(s) nuevo(s)`. En la bandeja de Beto, el aviso de 30 aparece **una sola vez** (id 21) arriba, y más abajo siguen los dos repetidos de antes del arreglo (`por 20`, ids 19 y 17). Esa lista junta el antes y el después.

```
Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s09 (trabajo-s09)
$ docker compose up -d --wait notificacion
[+] up 4/4
 ✔ Container mexi-banco-v06-discovery-1    Healthy                                                                1.2s
 ✔ Container mexi-banco-v06-db-1           Healthy                                                                1.2s
 ✔ Container mexi-banco-v06-rabbitmq-1     Healthy                                                                1.2s
 ✔ Container mexi-banco-v06-notificacion-1 Healthy                                                               15.7s

Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s09 (trabajo-s09)
$ curl -s 'http://localhost:8080/api/notificaciones?clabe=002180000000000002' | cut -c1-500
[{"id":21,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 30, saldo resultante 786.00","fecha":"2026-11-07T04
:55:03.249300Z"},{"id":19,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 20, saldo resultante 756.00","fecha
":"2026-11-07T02:34:15.652602Z"},{"id":17,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 20, saldo resultant
e 756.00","fecha":"2026-11-07T02:34:15.529390Z"},{"id":13,"clabeCuenta":"002180000000000002","mens

Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s09 (trabajo-s09)
$ echo


Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s09 (trabajo-s09)
$ docker compose logs notificacion | grep "consumidor\|ya procesado" | tail -4 | cut -c1-330
notificacion-1  | 2026-11-07T04:55:03.427Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.m.ConsumidorDeTransferencias       : [co
nsumidor] TransferenciaRealizada de la transferencia 16 (evento 6f450509-9943-46b0-95c1-3d94984f3c54, redelivered=true): 2 aviso(s) nu
evo(s)
notificacion-1  | 2026-11-07T04:55:03.435Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.notificacion.NotificacionService     : [no
tificacion] evento 6f450509-9943-46b0-95c1-3d94984f3c54 ya procesado: se ignora
notificacion-1  | 2026-11-07T04:55:03.436Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.m.ConsumidorDeTransferencias       : [co
nsumidor] TransferenciaRealizada de la transferencia 16 (evento 6f450509-9943-46b0-95c1-3d94984f3c54, redelivered=true): 0 aviso(s) nu
evo(s)
```

Figura 17. *Después del arreglo:* `ya procesado: se ignora` *y* `0 aviso(s) nuevo(s)` *en el log; un solo* `TRANSFERENCIA_RECIBIDA por 30` *en la bandeja.*

Además dejé dos pruebas unitarias nuevas en `NotificacionServiceTest`: un evento nuevo guarda los dos avisos, y un evento repetido no guarda nada (y no llama a `saveAll`). Las dos pasan (Anexo B).

5. La DLQ

Un mensaje que **nunca** se va a poder procesar (un *poison pill*) no se arregla por más que se reintente. Si se queda en la cola, estorba. Lo mandé a una **dead letter queue** (DLQ), donde una persona lo pueda revisar.

5.1 El "antes": el mensaje malo

Publiqué desde el panel, con la routing key `transferencia.realizada`, un Payload que no es un evento válido: `{"hola":`. El consumidor lanza una excepción al leerlo (`JsonEOFException: Unexpected end-of-input`) y **nunca llega al basicAck**.

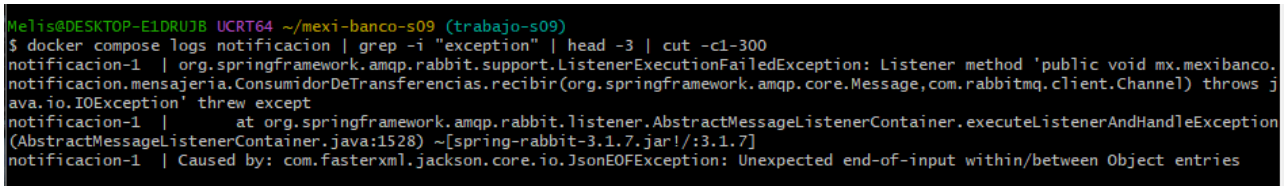


Figura 18. *Log de `notificacion`: `ListenerExecutionFailedException` en `ConsumidorDeTransferencias.recibir`, causada por `JsonEOFException: Unexpected end-of-input`.*

Lo que observé:

- El mensaje **no rebota en un ciclo**: se queda **trabado en Unacked** (con ack manual, el broker lo da por entregado y espera una respuesta que nunca llega).
- El consumidor **no se cae**: una transferencia nueva (de 5) siguió generando su aviso en la bandeja de Beto.
- Pero ese mensaje ocupa para siempre **uno de los 5 lugares del prefetch**. Con 5 mensajes así, el consumidor se quedaría sin lugares y dejaría de recibir hasta los buenos.

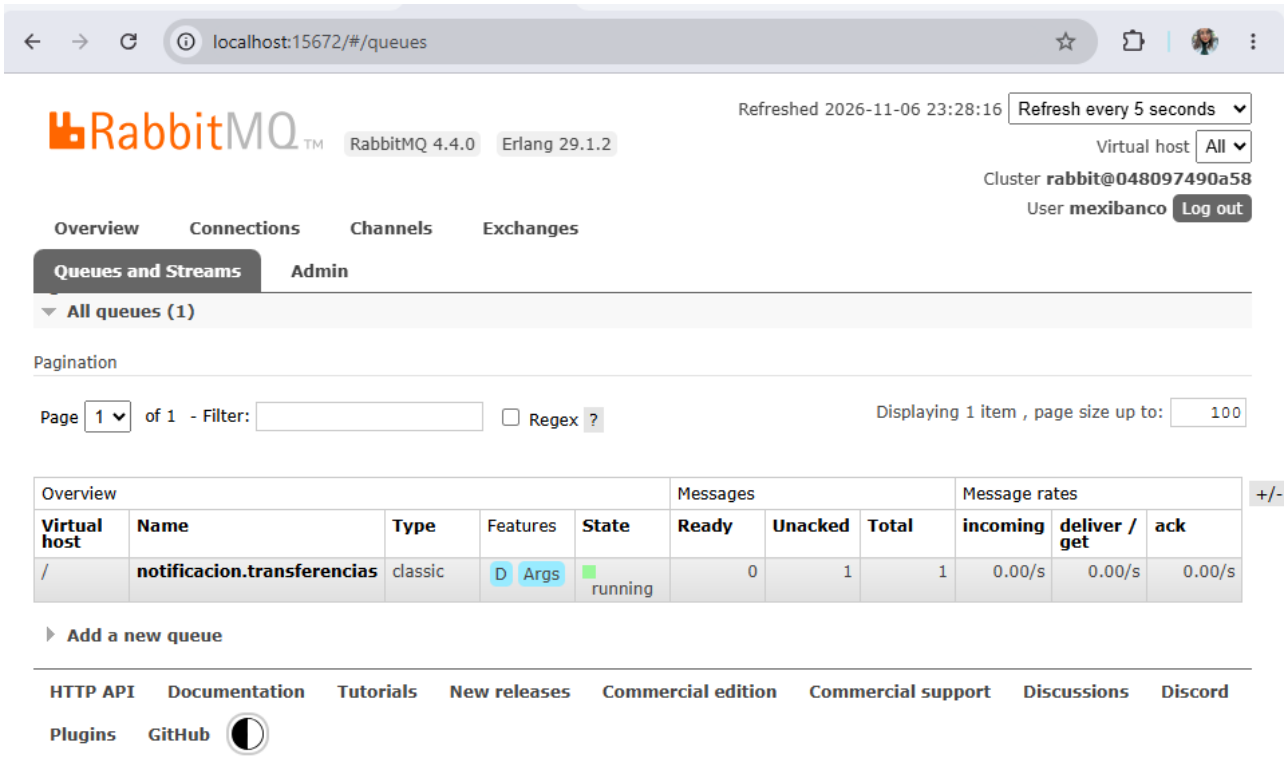


Figura 19. *Panel:* `notificacion.transferencias` *con* `Ready = 0` *y* `Unacked = 1`*: el mensaje malo, trabado.*

```
Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s09 (trabajo-s09)
$ docker compose exec rabbitmq rabbitmqctl list_queues name messages_ready messages_unacknowledged
Timeout: 60.0 seconds ...
Listing queues for vhost / ...
name    messages_ready  messages_unacknowledged
notificacion.transferencias     0       1

Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s09 (trabajo-s09)
$ curl -s -o /dev/null -w '%{http_code}\n' -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"
claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":5}'
201
```

Figura 20. *La cola con* `0` *Ready y* `1` *Unacked, y una transferencia nueva de 5 que responde* `201`*.*

```
Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s09 (trabajo-s09)
$ curl -s 'http://localhost:8080/api/notificaciones?clabe=002180000000000002' | cut -c1-250
[{"id":23,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 5, saldo resultante 791.00","fecha":"2026-11-07T05:
26:27.808374Z"},{"id":21,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 30, saldo resultan

Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s09 (trabajo-s09)
$ echo
```

Figura 21. *La bandeja de Beto tiene el aviso* `TRANSFERENCIA_RECIBIDA por 5` *(id 23): el mensaje malo no bloquea a los buenos, por ahora.*

5.2 La configuración de la DLQ

Tres piezas, con los nombres del contrato. La **cola principal** lleva dos argumentos que dicen a dónde se va un mensaje rechazado; el **DLX** `transferencias.dlx` (`direct`, durable) y la **DLQ** `notificacion.transferencias.dlq` (durable) se enlazan con la clave `notificacion.transferencias`. Es el **broker** quien mueve el mensaje rechazado, no mi servicio.

```
Queue cola = QueueBuilder.durable(COLA)
        .deadLetterExchange(DLX)              // x-dead-letter-exchange
        .deadLetterRoutingKey(COLA)           // x-dead-letter-routing-key
        .build();
DirectExchange dlx = ExchangeBuilder.directExchange(DLX).durable(true).build();
Queue dlq = QueueBuilder.durable(DLQ).build();
return new Declarables(transferencias, cola,
BindingBuilder.bind(cola).to(transferencias).with(PATRON),
        dlx, dlq, BindingBuilder.bind(dlq).to(dlx).with(COLA));
```

En el consumidor, lo que nunca se podrá procesar se rechaza con **basicNack sin reencolar** (el último `false` es `requeue`). Lo hice para tres casos:

- **JSON ilegible** (el caso de `{"hola":`).
- **JSON válido, pero sin eventoId o sin tipo:** tampoco se puede procesar nunca.
- **tipo desconocido:** el servicio lanza `IllegalArgumentException`.

```
try {
        evento = json.readValue(mensaje.getBody(), EventoTransferencia.class);
} catch (IOException ilegible) {
        log.error("[consumidor] mensaje ilegible, se manda a la DLQ: {}",
ilegible.getMessage());
        canal.basicNack(etiqueta, false, false);   // requeue = false: va al DLX
        return;
}
```

> **Una falla de mi base de datos no está en esa lista, a propósito.** Ahí el mensaje está bien y el problema es del servicio, no del mensaje: mandarlo a la DLQ lo sacaría de la cola para siempre por algo que se arregla. Ahí conviene dejarlo sin confirmar o reencolarlo, para que se reintente cuando la base vuelva.

5.3 El problema de PRECONDITION_FAILED

Mi cola de la sección 3 ya existía **sin** argumentos de DLX, y RabbitMQ no deja cambiar los argumentos de una cola existente. Al reconstruir `notificacion`, el log mostró este error repetido, y la DLQ nunca se creó (en el panel seguía habiendo una sola cola):

```
reply-code=406, reply-text=PRECONDITION_FAILED - inequivalent arg 'x-dead-letter-exchange'
for queue 'notificacion.transferencias' in vhost '/': ...
```

La solución es **borrar la cola** y volver a levantar el servicio para que la declare con los argumentos nuevos. Me salté un paso la primera vez (la cola no se había borrado de verdad) y repetí el borrado con el orden correcto: primero apagar `notificacion` (para que no recree la cola sin argumentos), luego borrar la cola y verificar que la lista quedara vacía, y al final levantar el servicio:

```
docker compose stop notificacion
docker compose exec rabbitmq rabbitmqctl delete_queue notificacion.transferencias
docker compose up -d --wait notificacion
```

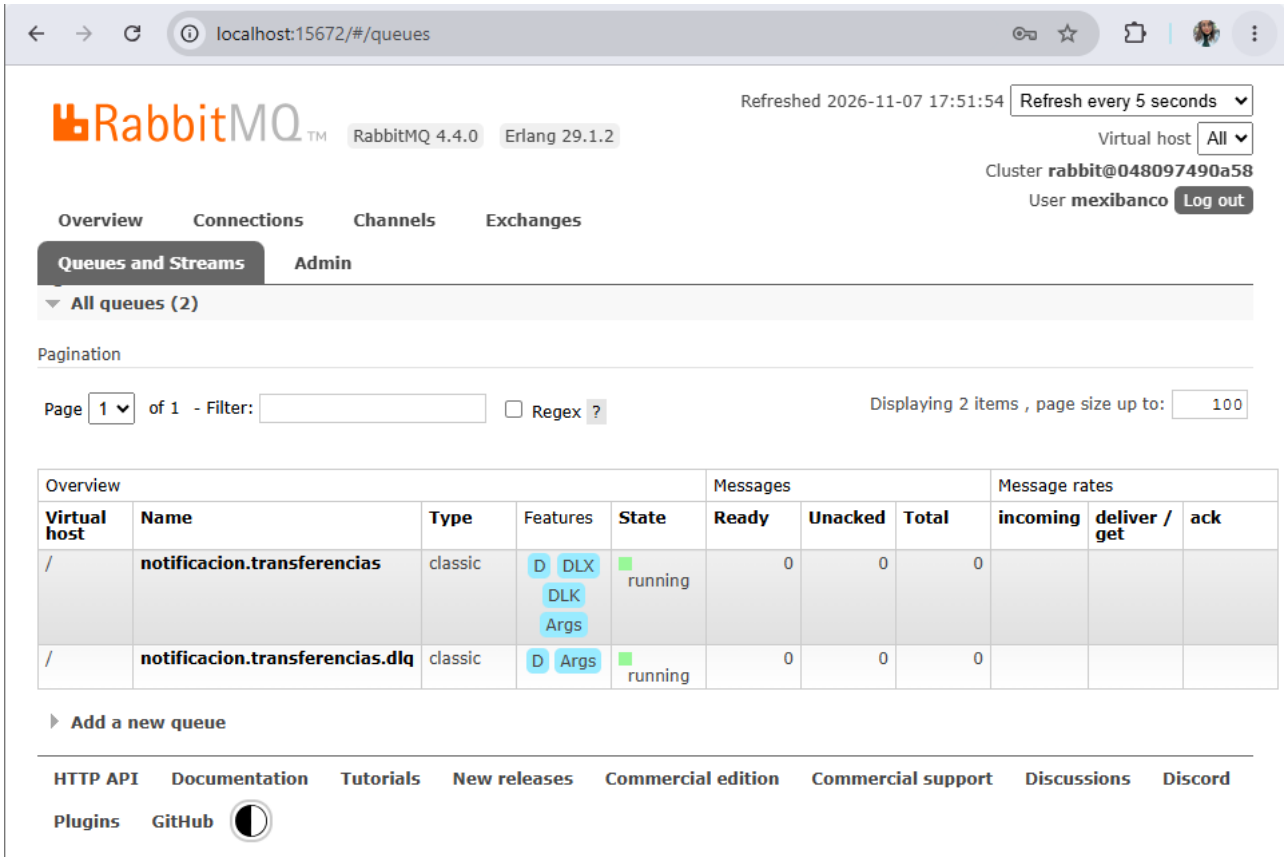


Figura 22. *Después de reconstruir: dos colas. La principal tiene las marcas `DLX` y `DLK` (dead letter exchange y routing key); la otra es `notificacion.transferencias.dlq`.*

5.4 El "después"

Publiqué otra vez `{"hola":`. Ahora el log dice `mensaje ilegible, se manda a la DLQ`, la cola principal queda en `0` y `0`, y la DLQ tiene **1 mensaje Ready**. Una transferencia nueva de 7 respondió `201` y llegó a la bandeja de Beto justo después (`por 7`, id 25): el mensaje malo ya no estorba.

```
Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s09 (trabajo-s09)
$ docker compose logs notificacion | grep "consumidor" | tail -2 | cut -c1-300
notificacion-1  | 2026-11-07T05:38:36.488Z ERROR 1 --- [notificacion] [ntContainer#0-1] m.m.n.m.ConsumidorDeTransferencias       : [co
nsumidor] mensaje ilegible, se manda a la DLQ: Unexpected end-of-input within/between Object entries
notificacion-1  | 2026-11-08T00:02:59.115Z ERROR 1 --- [notificacion] [ntContainer#0-1] m.m.n.m.ConsumidorDeTransferencias       : [co
nsumidor] mensaje ilegible, se manda a la DLQ: Unexpected end-of-input within/between Object entries

Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s09 (trabajo-s09)
$ docker compose exec rabbitmq rabbitmqctl list_queues name messages_ready messages_unacknowledged
Timeout: 60.0 seconds ...
Listing queues for vhost / ...
name    messages_ready  messages_unacknowledged
notificacion.transferencias.dlq 1       0
notificacion.transferencias     0       0

Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s09 (trabajo-s09)
$ curl -s -o /dev/null -w '%{http_code}\n' -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"
claveOrigen":"0021800000000000001","claveDestino":"0021800000000000002","monto":7}'
201

Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s09 (trabajo-s09)
$ sleep 6

Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s09 (trabajo-s09)
$ curl -s 'http://localhost:8080/api/notificaciones?clabe=0021800000000000002' | cut -c1-250
[{"id":25,"clabeCuenta":"0021800000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 7, saldo resultante 798.00","fecha":"2026-11-08T00:
07:49.608754Z"},{"id":23,"clabeCuenta":"0021800000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 5, saldo resultant

Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s09 (trabajo-s09)
$ echo
```

Figura 23. *Después del arreglo:* `mensaje ilegible, se manda a la DLQ`*, la DLQ con 1* `Ready`*, la transferencia nueva con* `201` *y su aviso* `por 7` *en la bandeja.*

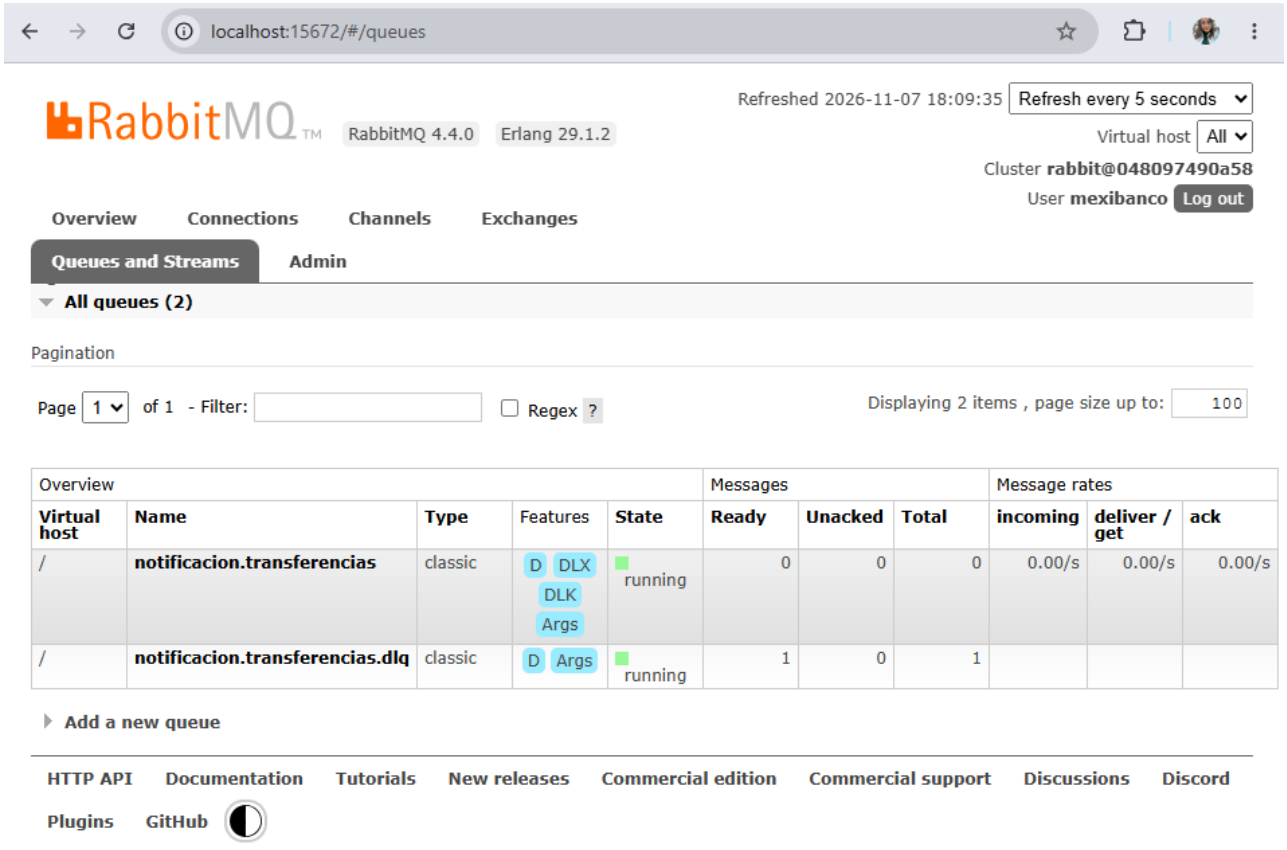


Figura 24. *Panel:* `notificacion.transferencias` *en* `0 / 0 / 0` *y* `notificacion.transferencias.dlq` *con* `1 Ready`*.*

Con **Get messages** en la DLQ se ve el mensaje apartado y la cabecera `x-death`: llegó por `transferencias.dlx`, `reason: rejected`, de la cola `notificacion.transferencias`.

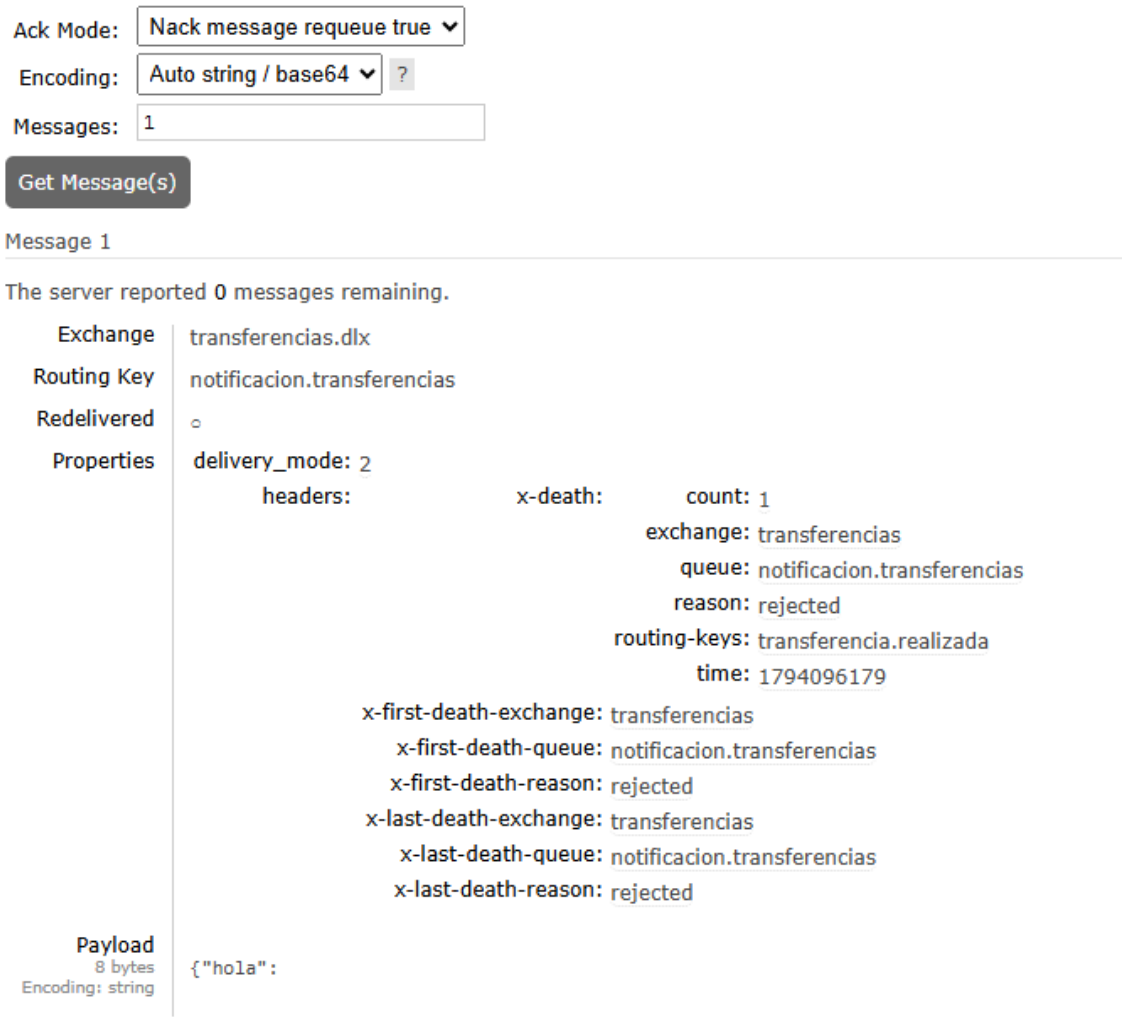


Figura 25. *El mensaje `{"hola":` dentro de la DLQ, con `x-death: reason: rejected, queue: notificacion.transferencias, routing-keys: transferencia.realizada`.*

Para pensar: ¿un JSON bien formado con un `tipo` que no existe, o sin `eventoId`, también es veneno? Sí: reintentarlo no lo arregla, así que también va a la DLQ (casos 2 y 3 del consumidor). ¿Y si lo que falló fue mi base de datos? Entonces el mensaje está bien y no lo mando a la DLQ: lo dejo para que se reintente cuando la base vuelva.

6. Preguntas de cierre

a) Si RabbitMQ está caído cuando la saga llega a COMPLETADA, ¿qué pasa con el aviso? ¿Por qué no lo arregla el publisher confirm?

En mi versión, **el aviso se pierde**. La saga guarda `COMPLETADA` en la base y después llama a `avisar()`; si RabbitMQ está caído, `publicar()` falla (a los 2 segundos del `connection-timeout`) y lanza `EventoNoPublicadoException`. `avisar()` la atrapa y solo deja un `ERROR` en el log ("quedó terminada pero el evento ... NO se publicó"). El cliente recibe su `201` porque el dinero sí se movió, pero nadie vuelve a intentar publicar el evento, y `notificacion` nunca se entera. No probé esto apagando RabbitMQ (era la prueba del Outbox, que no hice); lo deduzco del código y de la prueba de `NO_ROUTE` de la sección 2, que pasa por el mismo `catch`.

El **publisher confirm** no lo arregla porque la confirmación solo dice si el broker **recibió** el mensaje cuando se lo mandé; si el broker está caído, no hay nadie que me confirme y el mensaje solo existe en memoria. El problema es de fondo: es una **doble escritura**. La base dice `COMPLETADA` y el broker no se enteró, porque son dos sistemas sin una transacción que los una. La solución sería un **Transactional Outbox**: escribir el evento en una tabla de la **misma base**, en la misma transacción que `COMPLETADA`, y que un proceso aparte lo publique hasta que el broker confirme. No lo implementé.

b) ¿Por qué el ack va después de guardar y no antes? ¿Qué garantía obtienes con cada orden?

Imagino que `notificacion` muere justo entre las dos líneas, con cada orden:

Orden	Si el proceso muere entre las dos líneas	Garantía
Ack antes de guardar	El broker ya borró el mensaje y los avisos nunca se guardaron: el aviso **se pierde** y nadie lo reintenta.	**A lo sumo una vez**: nunca se repite, pero puede perderse.
Ack después de guardar (el mío)	Los avisos ya están en la base, pero el broker no recibió el ack y vuelve a entregar el mensaje (`redelivered=true`): el aviso **se repite**.	**Al menos una vez**: nunca se pierde, pero puede repetirse.

Elegí el segundo orden porque un aviso repetido **se puede detectar y descartar** con el `eventoId` (sección 4), y uno perdido no se puede recuperar. Por eso el ack después de guardar va acompañado de la idempotencia: juntos dan un efecto que se comporta como “exactamente una vez”.

c) Elige y di qué sacrificaste

> **Elección:** quité la llamada HTTP síncrona de `transferencia` a `notificacion` y publiqué un evento en RabbitMQ.
>
> **Porque:** `transferencia` ya no depende de que `notificacion` esté viva para terminar. En la Parte 0 una `notificacion` apagada dejaba la transferencia a medias y el cliente con un error aunque su dinero ya se había movido; ahora responde `201` y el aviso llega cuando `notificacion` vuelve (desacople en tiempo).
>
> **Sacrifiqué:** consistencia inmediata y simplicidad. El aviso ya no llega al instante, puede llegar repetido (por eso tuve que hacer el consumidor idempotente), puede quedar en la DLQ y, si RabbitMQ está caído justo en ese momento, se pierde sin el Outbox. Además hay más piezas que operar y entender (broker, cola, DLX, DLQ, tabla de eventos procesados).

Lo que no hice

No hice el **reto opcional del Outbox**. Su ausencia es justo la respuesta (a): sin él, un RabbitMQ caído en el momento de `COMPLETADA` pierde el aviso.

Anexo A. Dos errores que encontré en el camino

Los dos tuvieron la **misma causa raíz**: un enum guardado como texto y un constraint que Hibernate genera **una sola vez**, cuando crea la tabla. Mis bases venían de la S08 (mismo volumen de Postgres), así que ya tenían su tabla con el constraint viejo, y `ddl-auto: update` agrega columnas pero **no actualiza constraints**. Al agregar un valor nuevo al enum, la primera inserción con ese valor falla con un `500` sin explicación.

Constraint	Qué le faltaba	Cómo lo noté
`transferencia_estado_check` (base `transferencia`)	El estado `ABONADA`	En la Parte 0, la fila se quedó en `CARGADA` y el `500` no decía por qué. El log de `transferencia` mostró `Failing row contains (..., ABONADA, ...)` y `constraint [transferencia_estado_check].`
`movimiento_tipo_check` (base `movimiento`)	El tipo `DEVOLUCION_TRANSFERENCIA`	La compensación quedaba en `COMPENSACION_PENDIENTE`; `transferencia` decía `movimiento respondió 500` y el log de `movimiento` mostró `violates check constraint "movimiento_tipo_check".`

En los dos casos lo arreglé reemplazando el constraint en una sola transacción, sin tocar las filas:

```
docker compose exec db psql -U mexibanco -d transferencia -c "begin;
  alter table transferencia drop constraint transferencia_estado_check;
  alter table transferencia add constraint transferencia_estado_check check (estado in

('INICIADA','CARGADA','ABONADA','COMPLETADA','FALLIDA','COMPENSADA','COMPENSACION_PENDIENTE'
));
  commit;"

docker compose exec db psql -U mexibanco -d movimiento -c "begin;
  alter table movimiento drop constraint movimiento_tipo_check;
  alter table movimiento add constraint movimiento_tipo_check check (tipo in

('DEPOSITO','TRANSFERENCIA_ENVIADA','TRANSFERENCIA_RECIBIDA','SPEI_ENVIADO','DEVOLUCION_TRAN
SFERENCIA'));
  commit;"
```

Lo que aprendí: `ddl-auto: update` no es una migración. Si cambia un enum que se guarda como texto, el esquema no se actualiza solo y el error aparece lejos de la causa. En un proyecto real esto se resuelve con migraciones versionadas (Flyway o Liquibase), no con `ddl-auto`.

Anexo B. Pruebas unitarias

Las corrí con Maven dentro de un contenedor (`eclipse-temurin:21-jdk-alpine`, con `./mvnw test`). Los reportes de Surefire dan:

Clase de prueba	Pruebas	Fallas	Errores
`transferencia`: `BalanceoPorNombreTest`	1	0	0

Clase de prueba	Pruebas	Fallas	Errores
transferencia: CircuitoHaciaCuentaTest	4	0	0
transferencia: ClientesHttpTest	3	0	0
transferencia: TransferenciaServiceTest	10	0	0
notificacion: NotificacionServiceTest	3	0	0

En `TransferenciaServiceTest` cambié el `@Mock NotificacionCliente` por `@Mock PublicadorDeEventos`: el camino feliz ahora hace `verify(notificaciones).publicar(any())` (un evento, no dos llamadas) y la prueba de la compensada también espera ese `publicar`, porque ahora se publica `TransferenciaRevertida`. Las pruebas donde nunca se llega a publicar (cuenta inexistente, misma cuenta, cargo rechazado) siguen exigiendo `verifyNoInteractions(notificaciones)`. En `NotificacionServiceTest` agregué dos pruebas: un evento nuevo guarda los dos avisos y un evento repetido no guarda nada.

Anexo C. Qué cambié

Archivo	Cambio
transferencia/.../Transferencia.java	Campo `eventoId` (`UUID`, único), asignado en el constructor y con `@PostLoad` para las filas viejas.
transferencia/.../mensajeria/EventoTransferencia.java	Nuevo: el contrato del evento (`TransferenciaRealizada` / `TransferenciaRevertida`) y su routing key.
transferencia/.../mensajeria/ConfiguracionMensajeria.java	Nuevo: exchange `transferencias` (topic, durable) y convertidor JSON.
transferencia/.../mensajeria/PublicadorDeEventos.java	Nuevo: publica con entrega persistente y espera la confirmación del broker.
transferencia/.../mensajeria/EventoNoPublicadoException.java	Nuevo: el evento no se pudo publicar.
transferencia/.../TransferenciaService.java	Publica en lugar de llamar a `notificacion`; método `avisar()`; publica la revertida en `compensar()`.
transferencia/src/main/resources/application.yml	Publisher confirms, `mandatory`, `connection-timeout` y `espera-confirmacion-ms`.
transferencia/src/test/.../TransferenciaServiceTest.java	El mock ahora es `PublicadorDeEventos`; la compensada espera la publicación.

Archivo	Cambio
`notificacion/.../mensajeria/ConfiguracionMensajeria.java`	Nuevo: cola `notificacion.transferencias` con DLX, DLX `transferencias.dlx` y DLQ.
`notificacion/.../mensajeria/ConsumidorDeTransferencias.java`	Nuevo: consumidor con ack manual después de guardar y `basicNack` sin reencolar para lo ilegible.
`notificacion/.../EventoTransferencia.java`	Nuevo: copia local del contrato.
`notificacion/.../EventoProcesado.java` y `EventoProcesadoRepository.java`	Nuevos: tabla de eventos procesados y `marcarSiEsNuevo` (`INSERT ... ON CONFLICT DO NOTHING`).
`notificacion/.../NotificacionService.java`	`avisarDeTransferencia()` con la guarda de idempotencia.
`notificacion/src/main/resources/application.yml`	`acknowledge-mode: manual` y `prefetch: 5`.
`notificacion/src/test/.../NotificacionServiceTest.java`	Dos pruebas nuevas de idempotencia.